

Web Technologies

Web programming (I):
HTTP protocol
CGI scripts



Prof. Sabin Corneliu Buraga – profs.info.uaic.ro/sabin.buraga/
Assoc. Prof. Andrei Panu – profs.info.uaic.ro/andrei.panu/

“There are 2 ways to write error-free programs;
only the third one works.”

Alan Perlis

- ✨ the interaction between Web client(s) and server(s)
- 🥗 what is the architecture of the Web
- 🔧 how do we develop applications on the back-end



What is the Web?

World Wide Web

an information space containing
elements of interest, called **resources**,
denoted by global identifiers – **URI/IRI**

details at www.w3.org/TR/webarch/

W3C Recommendation (2004)

Web resources

Aspects of interest

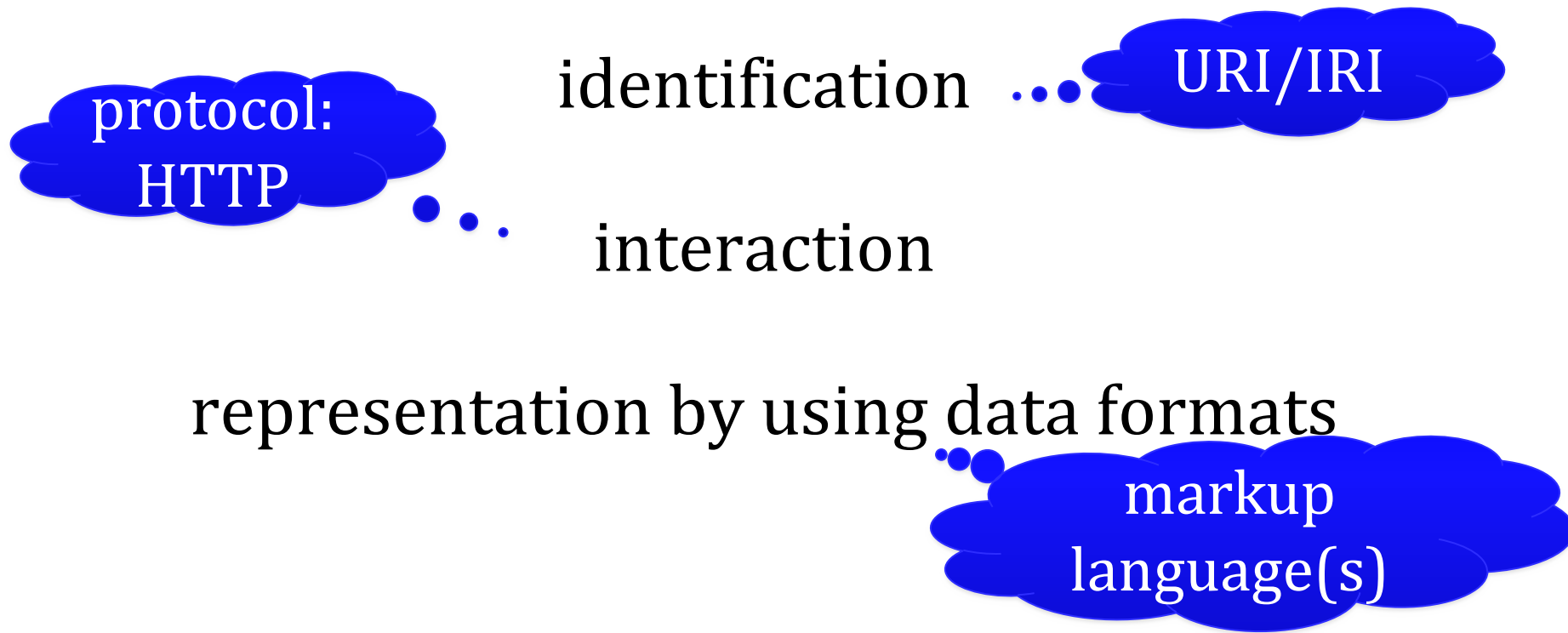
identification

interaction

representation by using data formats

Web resources

Aspects of interest





How about the interaction
between client(s) and Web server(s)?

HTTP

HyperText Transfer Protocol

based on the TCP/IP stack

HTTP

situated on the application layer

hypertext/hypermedia transfer
(HTTP – HyperText Transfer Protocol)

reliable transport via sockets
(TCP – Transmission Control Protocol)

network interconnection + data routing
(IP – Internet Protocol)

access control to the data transmission
medium (MAC – Medium Access Control)

HTTP

HyperText Transfer Protocol

a reliable request/response protocol

standard access port: **80**

HTTP

HTTP/1.1

Internet standard: RFC 2616 (1999)

current specifications: RFC 9110—9112

httpwg.org/specs/

resources of interest:

devdocs.io/http/

http.dev

HTTP

HTTP/2.0

RFC 9113 (2022)

focused on performance: binary messages, TCP connection reuse (a single connection per host), multiplexing (many parallel streams), sending messages to the client (server push), header compression – HPACK

<http2.github.io>

advanced

HTTP/2

HTTP/1.1

resources of interest:

<http2-explained.haxx.se>

www.digitalocean.com/community/tutorials/http-1-1-vs-http-2-what-s-the-difference

HTTP

HTTP/3.0

HTTP over QUIC (Quick UDP Internet Connections)

UDP, data stream multiplexing, data encryption, quick connection establishment – proposed in RFC 9114

explanations:

http.dev/3

www.smashingmagazine.com/2021/08/http3-core-concepts-part1/

http3-explained.haxx.se

HTTP: architecture

Web Server

daemon – “protective spirit”

Web Client

Web browser, robot (Web crawler), multimedia player, household appliance, conversational assistant, native application (desktop, mobile)...



Web Server

Apache HTTP Server, Eclipse Jetty, Gunicorn, Lighttpd, Microsoft Internet Information Services, NGINX, etc.

HTTP: architecture

Web Client

Web browsers: **Mosaic** ▶ **Netscape** ▶ **Mozilla.org** ▶ **Firefox**,
Internet Explorer ▶ **Edge**, **Chromium**, Safari, Opera GX etc.

tools: **curl**, **wget**, etc.

search engines robots – e.g., **Googlebot**

native applications offering various functionalities
implemented by service providers: e-commerce, finance,
tourism, entertainment, education, etc.

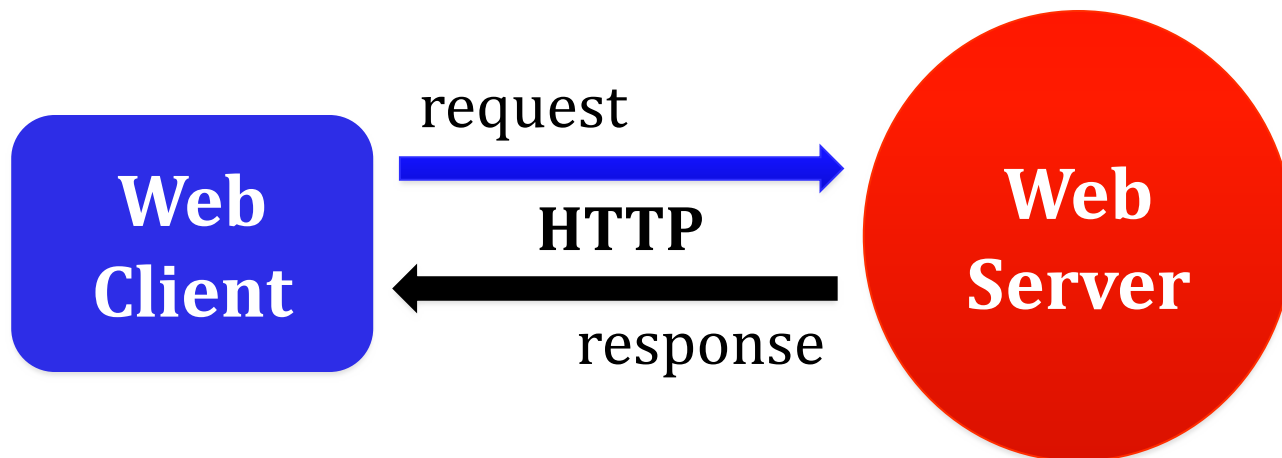
conversational and/or content generation systems

details: profs.info.uaic.ro/sabin.buraga/teach/courses/staw/web-film.html#week5

HTTP

Request and response

accessing – possibly, changing – a resource
representation by using its URI



HTTP: terms

Message

base unit of the HTTP communication
(request or response)

HTTP: terms

Intermediary



proxy



gateway



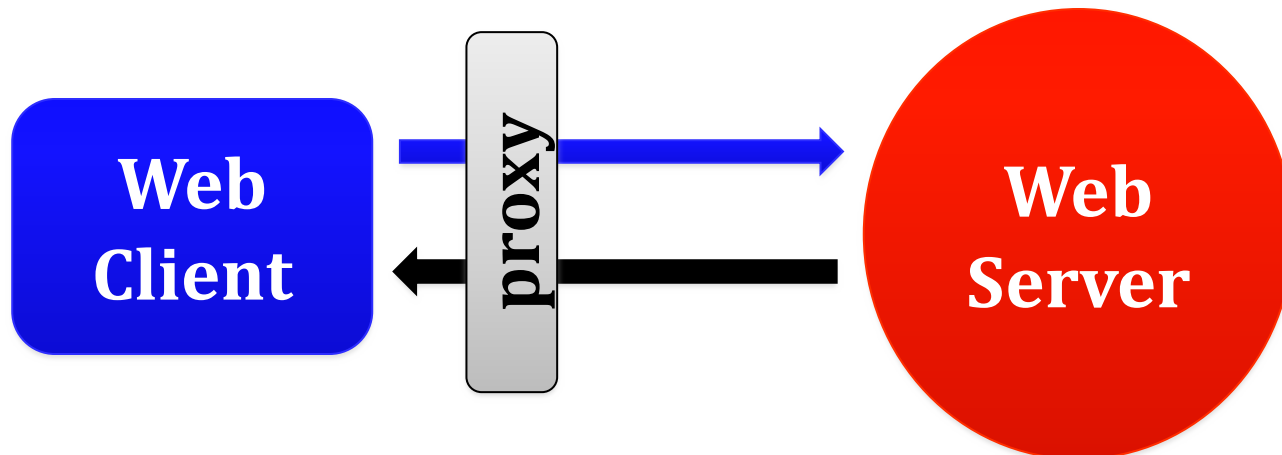
tunnel

HTTP: terms



Proxy

located in the client/server proximity
having the role of both server and client



HTTP: terms



Proxy

forward proxy

intermediary for a group of nearby clients
acts on behalf of clients

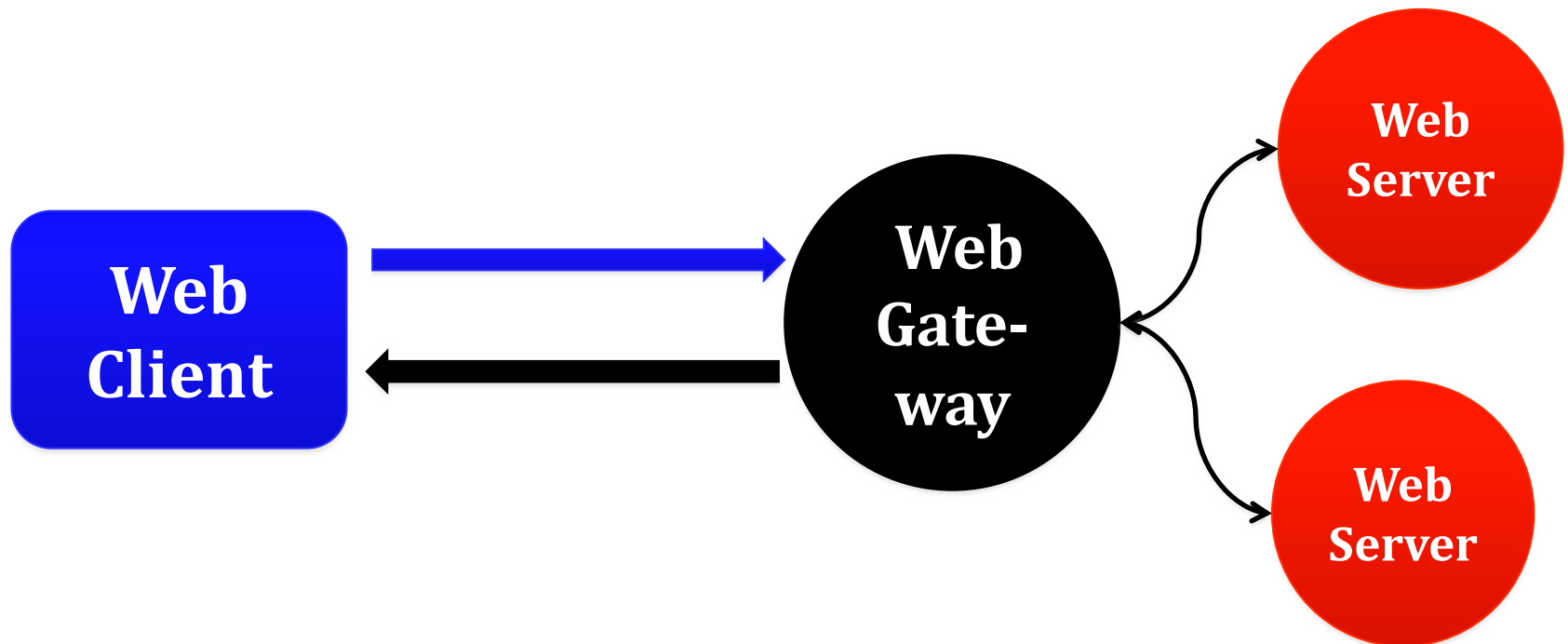
reverse proxy

intermediary for a group of nearby servers

HTTP: terms

Gateway

intermediary hiding the target (origin) server
the client has no knowledge about this



HTTP: terms

Gateway

can ensure:

- traffic distribution across servers – load balancing
- short-term data storage – caching
- message or request translation (e.g., HTTPS ► HTTP)
- other negotiation operations – role of mediator/broker

HTTP: terms

Gateway

open source software:

Apache Traffic Server – trafficserver.apache.org

HAProxy – www.haproxy.org

Squid – www.squid-cache.org

Vinyl Cache (ex-Varnish) – vinyl-cache.org

in cloud: Amazon ELB (Elastic Load Balancing)

aws.amazon.com/elasticloadbalancing/

HTTP: terms



retransmits – usually, encrypted – HTTP messages

HTTP: terms



Tunnel

context:

HTTPS protocol – ensures “secure” communication
HTTP via TLS (Transport Layer Security) at port 443

authentication based on digital certificates
+ bidirectional data encryption

visual tutorial: howhttps.works

Details about an HTTPS connection offered by a browser's developer tools

advanced

used encryption

information about the digital certificate

▼ Connection:

Protocol version: "TLSv1.3"

Cipher suite: "TLS_AES_256_GCM_SHA384"

Key Exchange Group: "x25519"

Signature Scheme: "RSA-PSS-SHA256"

▼ Host info.uaic.ro:

HTTP Strict Transport Security: "Disabled"

Public Key Pinning: "Disabled"

▼ Certificate:

▼ Issued To

Common Name (CN): "info.uaic.ro"

Organization (O): "Universitatea Alexandru Ioan Cuza din Iași"

Organizational Unit (OU): "<Not Available>"

▼ Issued By

Common Name (CN): "GEANT TLS RSA 1"

Organization (O): "Hellenic Academic and Research Institutions CA"

Organizational Unit (OU): "<Not Available>"

▼ Period of Validity

Begins On: "Fri, 21 Nov 2025 13:17:36 GMT"

Expires On: "Sat, 21 Nov 2026 13:17:36 GMT"

▼ Fingerprints

SHA-256 Fingerprint: "8D:F1:71:66:F4:7B:1B:6E:A5:A2...0:EB:3E:A9:DF:80:E3:BB:83:21"

SHA1 Fingerprint: "27:AA:16:82:E2:EE:A4:27:38:D1:21:A9:17:BA:D5:21:3B:F3:7A:45"

Transparency: "Valid SCT records"

HTTP: terms

Cache

local storage area – in memory, on disc –
for the messages (data)

server- and/or client-side

HTTP: terms

Cache

local storage area – in memory, on disc –
for the messages (data)

future requests for that data can be served faster

context: ensuring Web applications' performance

HTTP: messages

HTTP message = **header** + **body**

HTTP: messages

Header

includes a set of fields

field-name ":" [**field-value**] **CRLF**

CR = *Carriage Return* \r – code 13

LF = *Line Feed* \n – code 10

HTTP: messages

HTTP request

Method **Request-URI** **ProtocolVersion** **CRLF**
[**Message-header**] [**CRLF MIME-data**]

GET /web-technologies/ HTTP/1.1 **CRLF**
Host: edu.info.uaic.ro

HTTP: messages

HTTP response

HTTP-version Digit Digit Digit Reason
CRLF Content

HTTP/1.1 200 OK CRLF ...

HTTP: methods

GET

request – performed by a client – to access
a resource's representation

HTTP: methods

GET

request – performed by a client – to access
a resource's representation

HTML document, CSS stylesheet,
image in JPEG/PNG format, vector illustration as SVG,
JavaScript program, data in JSON (JavaScript Object
Notation) format, 3D model, RSS (XML) news feed,
PDF presentation, ZIP archive, video, ...

HTTP: methods

HEAD

similar to GET
usually, offers only meta-data

HTTP: methods

HEAD

similar to GET

usually, offers only meta-data

e.g., MIME type (Media Type) of a resource, last update,...

HTTP: methods

PUT

updates a resource representation or, eventually,
creates a resource on the Web server

A blue, cloud-like bubble with a soft shadow, containing white text. It has three small circles at the top right, suggesting a trail or movement.

details in the lecture
regarding Web services

HTTP: methods

POST

creates a resource, usually sending entities (data, actions) to the server

HTTP: methods

POST

creates a resource, usually sending entities (data, actions) to the server

e.g., data entered into a Web form's fields

HTTP: methods

DELETE

erases a resource – its representation –
from the server

HTTP: methods

Remark

traditionally, the Web browser
only permits the use of GET and POST methods

HTTP: methods

A method is considered **safe**
if it does not modify the server state
i.e. no side-effect actions are performed on the server

GET and HEAD are safe

POST, PUT, and DELETE are not safe

details in the lecture
regarding Web services

HTTP: methods

A method is considered **idempotent** when it can be called many times without different outcomes, returning the same response (representation)

GET, HEAD, PUT, and DELETE are idempotent

POST is not idempotent

A blue, cloud-like bubble with a soft shadow, containing white text. It has three small circles at the top right, suggesting a trail or movement.

details in the lecture
regarding Web services

HTTP: resource representations

Character set encodings

ISO-8859-1

ISO-8859-2

KOI8-R

ISO-2022-JP

UTF-8

UTF-16 Little Endian

...

HTTP: resource representations

Message (content) encodings

compression, identity, and/or integrity

traditional approach: gzip – gzip.org

modern approaches:

Brotli – brotli.org (Google)

zstd – facebook.github.io/zstd/ (Facebook)

HTTP: resource representations

Representation formats

text

HTML, CSS, plain text, JavaScript code, XML document

or

binary

image, PDF document, multimedia resource, .gzip archive

HTTP: resource representations

Resource's content type

media types

HTTP: header fields (attributes)

Content-Type

permits the transfer of any kind of data

Content-Type: type/subtype

HTTP: header fields (attributes)

Content-Type

specified by **Media Types – MIME**
(**Multipurpose Internet Mail Extensions**)

denotes a set of **primary content types**
+ additional **sub-types**

initially, used in the e-mail context

HTTP: header fields (attributes)

MIME types:



text indicates textual formats

text/plain – unformatted text

text/html – HTML (HyperText Markup Language) document

text/css – CSS (Cascading Style Sheets) resource

HTTP: header fields (attributes)

MIME types:



image specifies graphical formats

image/gif – GIF (Graphics Interchange Format) images

image/jpeg – JPEG (Joint Picture Experts Group) photos

image/png – PNG (Portable Network Graphics) pictures

image/webp – WebP (Web Picture Format) images

image/avif – (animated) images optimized for the Web

image/svg+xml – SVG (Scalable Vector Graphics) illustrations

HTML **** – developer.mozilla.org/docs/Web/HTML/Element/img

HTTP: header fields (attributes)

MIME types:



audio denotes audio content

audio/mpeg – resource encoded in MP3 format

specification for audio data according to the MPEG (Motion Picture Experts Group) standard – tools.ietf.org/html/rfc3003

audio/ac3 – compressed audio resource
conforming to AC-3 standard – www.atsc.org/standards/

HTML **<audio>** – developer.mozilla.org/docs/Web/HTML/Element/audio

HTTP: header fields (attributes)

MIME types:

 **video** defines video content: animations, films

video/av1 – open format: Alliance for Open Media

aomedia.org/av1-features/

video/h265 – resource in High Efficiency Video Coding format

www.itu.int/rec/T-REC-H.265

W3C standard for real-time multimedia communication:

WebRTC (2025): www.w3.org/TR/webrtc/

HTML **<video>** – developer.mozilla.org/docs/Web/HTML/Element/video

HTTP: header fields (attributes)

MIME types:



application signifies formats that can be processed by applications on the client-side

application/epub+zip – EPUB e-book in ZIP format

application/javascript – JavaScript program

application/json – JSON (JavaScript Object Notation) data

application/octet-stream – stream of arbitrary bytes

application/senml+xml – XML data from sensors: RFC 8428

HTTP: header fields (attributes)

MIME types:

 **multipart** used to transfer composed data

multipart/mixed – mixed content

multipart/alternative – alternative contents



e.g., different qualities
of multimedia streams

N. Freed *et al.*, *Media Types* (17 February 2026)

www.iana.org/assignments/media-types/media-types.xhtml

calendar+json	application/calendar+json	Calendar in JSON format
csv	text/csv	CSV data
heic	image/heic	High Efficiency Image File
markdown	text/markdown	Text in Markdown format
msword	application/msword	Word (MS Office) document
vnd.rar	application/vnd.rar	Proprietary format
VP8	video/VP8	Video format VP8: RFC 7741
woff2	font/woff2	Typographic fonts format
x3d	model/x3d+xml	3D model in X3D XML format
zip	application/zip	ZIP archive

(instead of) break

Today

Are you https? Because I'm ://
without you

Nah, I'm just plain old HTTP
because I'm insecure

Delivered

HTTP: header fields (attributes)

Location

Location ":" "http(s)://" authority [":" port] [abs_path]

redirects the client to another resource representation
(HTTP redirect)

Location: <http://somewhere.info:8080/moved.html>

HTTP: header fields (attributes)

Referer

denotes the URI of a Web resource that refers to the current resource

used to know the source of the requests to a given document (back-links) for analytics, logging, caching,...

HTTP: header fields (attributes)

Host

specifies the target address – IP or symbolic domain – of the machine supposed to provide a requested resource

HTTP: header fields (attributes)

Other existing fields concern the following:

- accepted content (content negotiation) – e.g., Accept
- authentication & authorization – WWW-Authenticate Authorization
- conditional access to resources – If-Match, If-Modified-Since,...
- cache – Age, Cache-Control, Expires, ETag, etc.
- proxy – Proxy-Authenticate, Proxy-Authorization, Via
- message delivery (HTTP push) – Topic, TTL, Urgency
- ...and many many others

developer.mozilla.org/docs/Web/HTTP/Headers

HTTP: status

Informational (1xx)

100 Continue, 101 Switching Protocols, 102 Processing

Status	Method	Domain	File	Initiator	Type	Transferred
101	GET	 public-api.wordpress.com	newest-note-data	websocket	plain	244 B

switching protocol

here, from HTTP to WebSocket

www.tutorialspoint.com/websockets/

HTTP: status

Success (2xx)

200 Ok, 201 Created, 202 Accepted,
204 No Content, 206 Partial Content,...

200	OPTIONS	 people-pa.clients6.g...	people?person_id= xhr	html	681 B
200	POST	 clients6.google.com	v2internal?\$ct=mu m=b:353 (xhr)	mixed	1.88 kB
200	GET	 people-pa.clients6.g...	people?person_id= m=b:353 (xhr)	json	7.63 kB

OPTIONS – method to determine server capabilities or requirements for a resource
the transfers are asynchronous (**xhr** – **XMLHttpRequest** is used)

204	POST	 docs.google.com	naLogImpressions?id=1i0F1p 1526657789-waffle_js_...	plain	6.43 kB
-----	------	---	---	-------	---------

HTTP: status

Redirection (3xx)

300 Multiple Choices, 301 Moved Permanently, 302 Found, 303 See Other, 304 Not Modified, 305 Use Proxy, etc.

Status	Method	Domain	File	Initiator	Type	Transferred
301	GET	tinyurl.com	fii-web-zoom	document	html	3.96 kB
200	GET	us02web.zoom.us	83547687804?pwd=bk1YT2EvaTh	document	html	6.75 kB
304	GET	static.ada.support	embed.js	script	js	cached
200	304 Not Modified	us02web.zoom.us	lres	script	js	cached
302	GET	www.slideshare.net	/busaco/	document	html	10.35 kB 37.79 kB
200	GET	www.slideshare.net	/busaco/	document	html	11.08 kB 37.79 kB
	GET	www.linkedin.com	noauthtracker?type=le img	Blocked By uBlock ...		
200	GET	public.slidesharec...	ss-core.css?bc41874	Slow server response time (1.06 s). The recommended limit is 500 ms.		

HTTP: status

Client Error (4xx)

400 Bad Request, 401 Unauthorized, 403 Forbidden, 405 Method Not Allowed, 408 Request Timeout, 410 Gone, 414 Request-URI Too Long, 415 Unsupported Media Type, 423 Locked, 429 Too Many Requests,...

400	GET	 i0.wp.com	Logo_design_jam_iasi.png	img	html
400	400 Bad Request		i0.wp.com	Logo_design_jam_iasi.png	html
401	GET	 api.nytimes.com	articlesearch.json?q=w&api-key=1	document	

HTTP: status

Server Error (5xx)

500 Internal Server Error, 501 Not Implemented,
502 Bad Gateway, 503 Service Unavailable,
505 HTTP Version Not Supported, 508 Loop Detected,...

501	PATCH	 httpbin.org	501	swagger-ui-bundle.js:72 (fetch)	html
5	501 Not Implemented	tpbin.org	501	swagger-ui-bundle.js:72 (fetch)	html

the transfers are asynchronous (**fetch** – **Fetch API** is used, offered by HTML5)

Bad gateway

Error code 502

Visit cloudflare.com for more information.

2025-02-02 16:51:01 UTC

advanced



You

Browser
Working



Bucharest

Cloudflare
Working



sgm.adem.my.id

Host
Error

What happened?

The web server reported a bad gateway error.

What

Please

504 Gateway Timeout ERROR

The request could not be satisfied.

We can't connect to the server for this app or website at this time. There might be too much traffic or a configuration error. Try again later, or contact the app or website owner. If you provide content to customers through CloudFront, you can find steps to troubleshoot and help prevent this error by reviewing the CloudFront documentation.

Generated by cloudfront (CloudFront) HTTP3 Server
Request ID: oacaJWUhszX0JmuyNISqP0kVehbzohnioZWKM_47k5gN0AhPaJRgQ==

Cloudflare offers content distribution services, ensuring performance and security of Web applications and has a role of reverse proxy, being located between the user's Web browser and the site hosted on the target Web server

HTTP: logging

Requests sent to a Web server are logged

Common Log Format

standardized text file format

for Apache HTTP Server: **mod_log_config** module

<httpd.apache.org/docs/current/logs.html>

NGINX: config via **access_log** directive

docs.nginx.com/nginx/admin-guide/monitoring/logging/

w10.uaic.ro - msi2019 [13/Feb/2020:14:53:14 +0200]
"GET /~vidrascu/MasterSI2/note/Restanta.pdf HTTP/1.1" 206 25227
"https://profs.info.uaic.ro/~vidrascu/MasterSI2/index.html" "...Chrome/79.0.3945.130"
82-137-8-231.rdsnet.ro - - [13/Feb/2020:15:38:23 +0200]
"POST /~computernetworks/login.php HTTP/1.1" 302 1115
"https://profs.info.uaic.ro/~computernetworks/login.php"
"...X11; Ubuntu; Linux x86_64 ... Firefox/73.0"
ec2-23-21-0-202.compute-1.amazonaws.com - - [13/Feb/2020:15:48:29 +0200]
"GET /~busaco/teach/courses/web/presentations/web01ArhitecturaWeb.pdf HTTP/1.1"
200 2081804 "-" "HTTP_Request2/2.3.0 (http://pear.php.net/package/http_request2)..."
199.16.156.126 - - [13/Feb/2020:15:58:58 +0200]
"GET /robots.txt HTTP/1.1" 404 182 "-" "Twitterbot/1.0"
psihologie-c-113.psih.uaic.ro - - [13/Feb/2020:16:03:04 +0200]
"GET /~busaco/ HTTP/1.1" 200 1942 "-" "... Firefox/72.0..."
psihologie-c-113.psih.uaic.ro - - [13/Feb/2020:16:03:04 +0200]
"GET /~busaco/csb.css HTTP/1.1" 200 852 "https://profs.info.uaic.ro/~busaco/"
"... Firefox/72.0..."
proxy-220-255-2-224.singnet.com.sg - - [13/Feb/2020:16:23:23 +0200]
"GET /favicon.ico HTTP/1.1" 200 1406 "-" "...UCBrowser/11.3.8.976..."
c2.uaic.ro - - [13/Feb/2020:16:33:43 +0200]
"GET /~busaco/teach/courses/web/ HTTP/1.1" 304 - "-" "...Chrome/80.0.3987.106"
220.181.51.219 - - [13/Feb/2020:19:20:20 +0200]
"HEAD /%7Ebusaco/music/09.Sabin%20Buraga%20-...mp3 HTTP/1.0" 200 - "-"
"NSPlayer/10.0.0.4072 WMFSDK/10.0"

GET /web-technologies/ HTTP/1.1
Host: edu.info.uaic.ro
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:135.0)
Gecko/20100101 Firefox/147.0
Accept: [text/html](#),[application/xhtml+xml](#),[application/xml](#);
q=0.9,[image/avif](#),[image/webp](#),[*/*](#);q=0.8
Accept-Language: en-GB,en;q=0.5
Accept-Encoding: gzip, deflate, br , zstd
Connection: keep-alive
Referer: <https://profs.info.uaic.ro/andrei.panu/>

HTTP: example of a request

HTTP/1.1 200 OK

Date: Fri, 20 Feb 2026 10:03:32 GMT

Server: Apache

Last-Modified: Thu, 19 Feb 2026 16:08:15 GMT

Content-Encoding: gzip

Content-Length: 1060

Keep-Alive: timeout=5, max=100

Connection: Keep-Alive

Content-Type: text/html; charset=UTF-8

header fields
(meta-data)

content

<!DOCTYPE html>

<html lang="ro">

...

...

</html>

HTTP: example of a response

GET /services/feeds/photos_public.gne?tags=Fll,lasi

Host: www.flickr.com

...

HTTP/2 200 OK

Content-Type: application/atom+xml; charset=utf-8

Date: Thu, 19 Feb 2026 08:48:10 GMT

Server: openresty

Expires: Mon, 26 Jul 1997 05:00:00 GMT

Last-modified: Sun, 02 Nov 2014 06:58:25 GMT

Cache-control: private, no-store, no-cache, must-revalidate

Pragma: no-cache

X-Cache: Miss from cloudfront

Via: 1.1 46d5c1a4d1e3a5c8a14bdb9b6676ba11.cloudfront.net (CloudFront)

X-Server: serverless-proxy-10.78.47.1

X-Firefox-Spdy: h2

advanced

data in Atom format
(processed by the client)

expires in the past
(won't be kept in cache)

X fields are not
standardized

obtaining information about public pictures offered by Flickr

HTTP: logging – HAR format

The interaction between the browser and the Web server (requests + responses) can be stored in HAR files (HTTP ARchive)

JSON based format

www.softwareishard.com/blog/har-12-spec/

example: gist.github.com/igrigorik/3495174

HTTP: logging – HAR format

The interaction between the browser and the Web server (requests + responses) can be stored in HAR files (HTTP ARchive)

main purpose: analyzing Web traffic

important aspect: performance

also see the HTTP Archive initiative:

httparchive.org

HTTP: APIs (libraries)

cURL + libcurl

(C, Java, Haskell, .NET, PHP, Ruby,...) – curl.haxx.se

Apache HttpComponents (Java) – hc.apache.org

HTTP (Node.js module) – nodejs.org/api/http.html

http.client (Python 3)

LibHTTP (C library): www.libhttp.org



details in the lecture
about Web services

HTTP: client-side tools

Google Chrome Developer Tools

developer.chrome.com/docs/devtools

Firefox Developer Tools

firefox-source-docs.mozilla.org/devtools-user/

Microsoft Edge DevTools

learn.microsoft.com/en-us/microsoft-edge/devtools/

additional information:

devtoolstips.org

Inspector Console Debugger Performance Network Storage DOM Application

Filter URLs

Disable Cache No Throttling

All HTML CSS JS XHR Fonts Images Media WS Other

Status	Method	Domain	File	Initiator	Type	Transferred	Size
304	GET	occ-0-3018-3466....	AAAABb89JNT487jNJ_mRF6AaGV3q9UWNzNxNKbsfF0suBLNc	img	jpeg	cached	41.04 KB
304	GET	occ-0-3018-3466....	AAAABeFa98bPMrT-JmX04T3-naWQ7ZH9kcmQRWHxaSSJN4C	img	jpeg	cached	25.01 KB
200	GET	codex.nflxext.com	none	script	js	3.96 KB	9.01 KB
200	GET	codex.nflxext.com	none	script	js	777.11 KB	2.62 MB
	GET	assets.nflxext.com	nficon2016.ico	FaviconLoader.jsm:1...	x-icon	16.56 KB (...)	16.56 KB
101	GET	push.prod.netflix.c...	ws	websocket	plain	175 B	0 B
	GET	ae.nflximg.net	adtech_iframe_target_05.html?data={"membership_status":"Cl	none:11 (subdocume...		Blocked B...	
200	POST	www.netflix.com	pathEvaluator?webp=false&drmSystem=widevine&isVolatileBillk	none:9 (xhr)	json	7.28 KB	28.72 KB
200	GET	www.netflix.com	probe?monotonic=false&device=web&iter=0	cadmium-playercore...	json	2.42 KB	1.12 KB
200	POST	www.netflix.com	pathEvaluator?webp=false&drmSystem=widevine&isVolatileBillk	none:9 (xhr)	json	2.48 KB	27 B
200	GET	nfrt-nrdp-nlb.prod....	probe?data=5120&device=web&ord=4&recipe=probnikNoFront	cadmium-playercore...	octet-stream	5.78 KB	5 KB
200	GET	nfrt-mobile-direct....	probe?data=5120&device=web&ord=0&recipe=probnikNoFront	cadmium-playercore...	octet-stream	5.79 KB	5 KB
200	GET	nfrt-mobile-nlb.pro...	probe?data=5120&device=web&ord=2&recipe=probnikNoFront	cadmium-playercore...	octet-stream	5.79 KB	5 KB
200	GET	nfrt-mobile-direct...	probe?data=5120&device=web&ord=1&recipe=probnikNoFronti	cadmium-playercore...	octet-stream	5.79 KB	5 KB
200	GET	nfrt-nrdp-direct.pr...	probe?data=5120&device=web&ord=3&recipe=probnikNoFront	cadmium-playercore...	octet-stream	5.79 KB	5 KB
304	GET	occ-0-3018-3466....	AAAABZSqcXJDH3nWFZk9HJx5FMAabIXDeFHovYYVlcDQg1d...	none:6 (img)	jpeg	cached	35.59 KB
304	GET	occ-0-3018-3466....	AAAABYIET22cXmL7amS5zxBdaguxTnVQspnUjsapLJyKXeOQC...	none:6 (img)	jpeg	cached	18.54 KB
304	GET	occ-0-3018-3466....	AAAABWdYKiWuRs8MbYQ_6TiWe8exVZZECeva_6tP7OiNCLkrk...	none:6 (img)	jpeg	cached	16.78 KB

100 requests 6.49 MB / 1.14 MB transferred Finish: 2.09 min DOMContentLoaded: 1.57 s load: 5.04 s

inspecting HTTP requests made by the browser



What is the architecture of the
Web server?

HTTP: Web server

Fulfills multiple requests from the clients using the HTTP protocol

HTTP: Web server

Fulfills multiple requests from the clients using the HTTP protocol

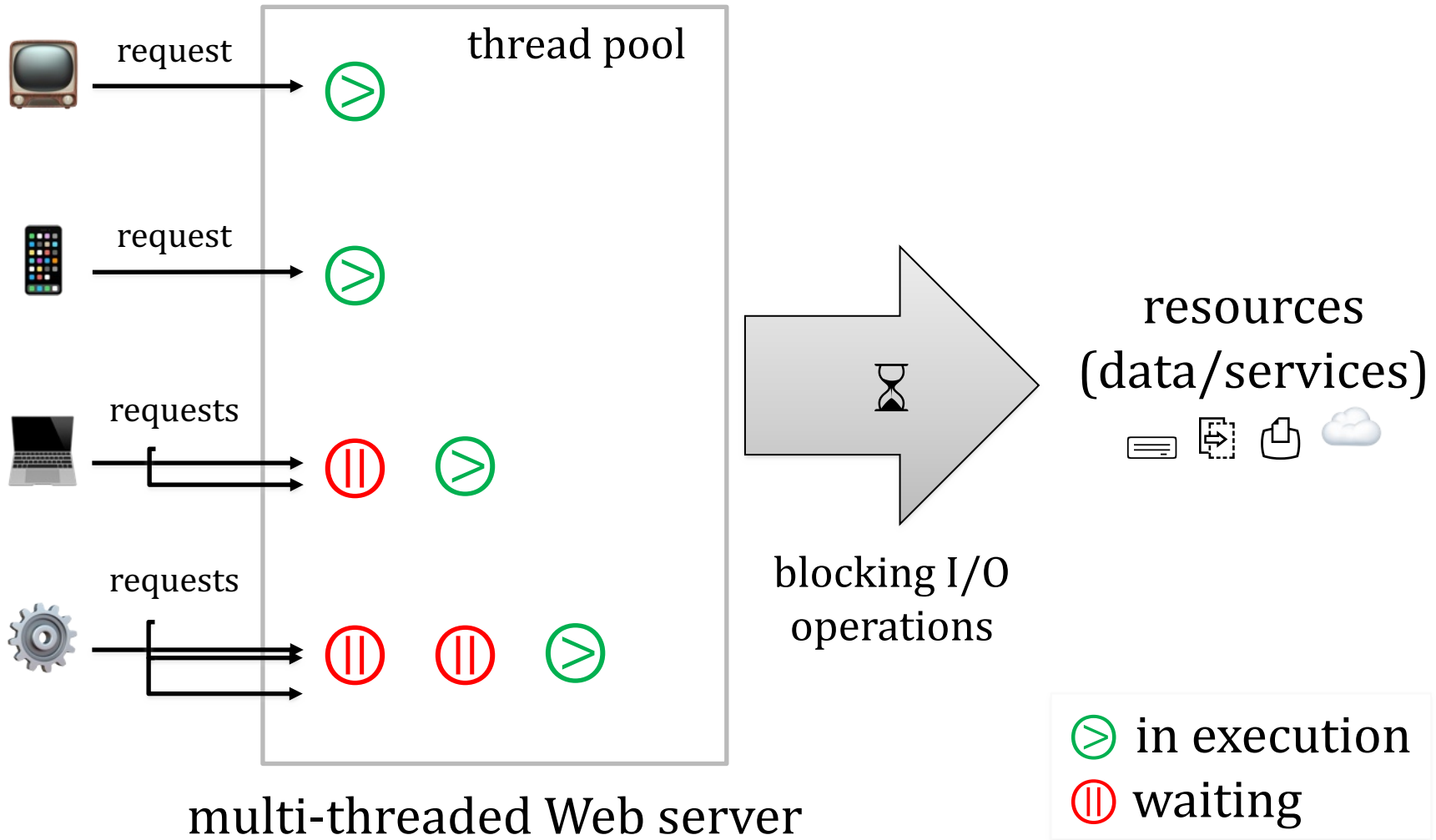
- ⚠ each request is considered independent from others, even though it comes from the same Web client
 - ▶ connection state is not kept – **stateless**

HTTP: Web server

Traditionally, the Web server implementation is either **pre-forked** or **pre-threaded**

on initialization, a number of child processes or threads are created, each process/thread interacting to a distinct client

example: **Apache HTTP Server Project**
<httpd.apache.org>



multiple requests from different clients cannot be served simultaneously
(the number of threads associated with a process is limited)

HTTP: Web server

Alternatively, asynchronous strategies can be used
(non-blocking), adopting a single thread
(single threaded)

reference example:

NGINX

blog.nginx.org/blog/inside-nginx-how-we-designed-for-performance-scale

HTTP: Web server

Alternatively, asynchronous strategies can be used
(non-blocking), adopting a single thread
(single threaded)

the programmer can create its own Web server, written in
JavaScript (ECMAScript) / TypeScript:

Node.js – nodejs.org

Deno – deno.com



How can we develop
the back-end of Web applications?

necessity

Dynamic generation – on the **server** –
of **representations** of resources
requested by clients

solutions

CGI – Common Gateway Interface

Web application servers

Web frameworks

solution: cgi

Language-independent programming interface
facilitating the interaction between clients and
programs invoked on the Web server

de facto standard

RFC 3875 – datatracker.ietf.org/doc/html/rfc3875
www.w3.org/CGI/

cgi

A CGI program (script) is invoked on server

explicit

i.e., retrieving data from a Web form
after the submit button is pressed

cgi

A CGI program (script) is invoked on server

implicit

example: at each visit a new ad (e.g., banner) is
generated

cgi

CGI scripts can be written in any language
available on the server

interpreted languages

bash, Perl – e.g., Perl::CGI module –, Python, Ruby,...

compiled languages

C, C++, Rust, etc.

cgi: programming

- Any CGI program will write data
- the representation of a Web resource –
at standard output (stdout)

cgi: programming

To denote the type of the generated representation,
HTTP headers are used – MIME (Media Types)

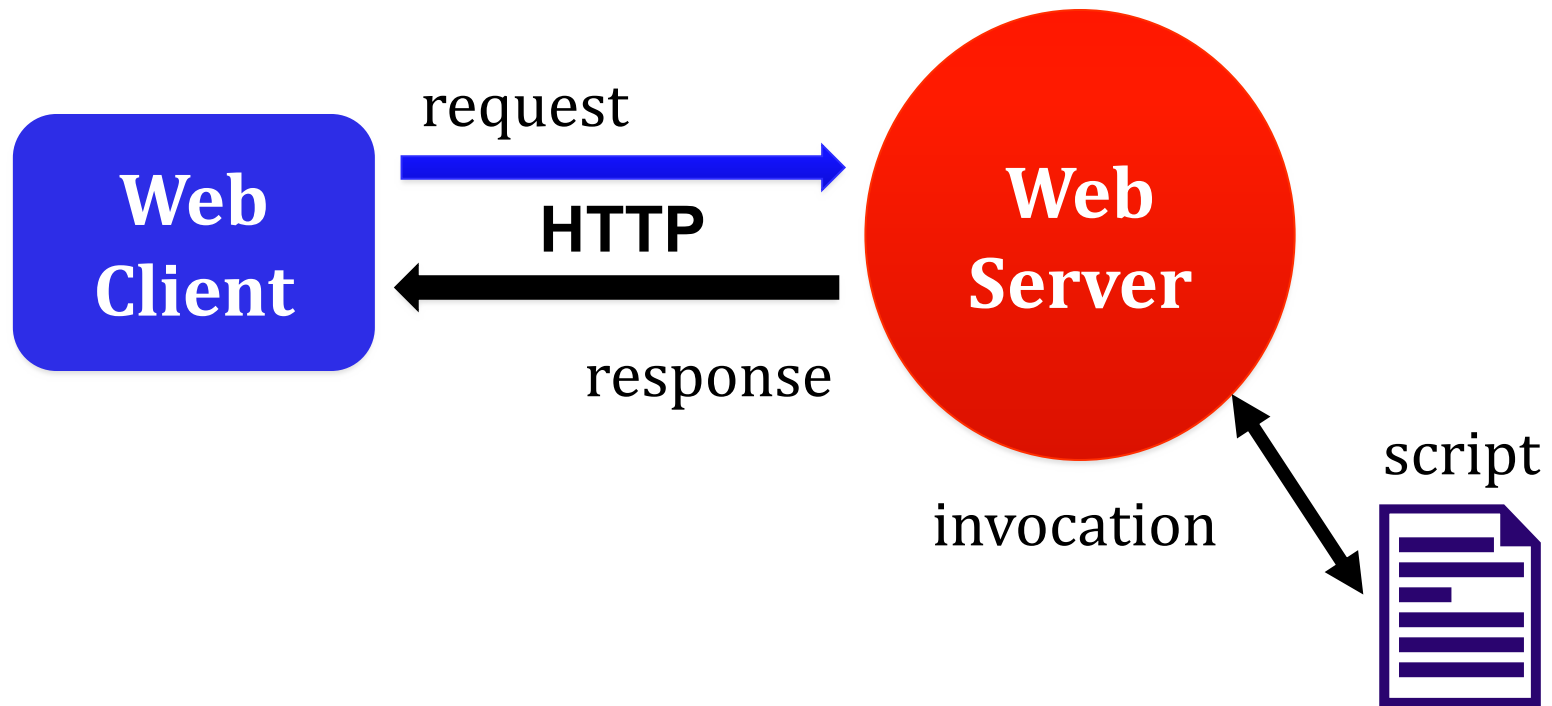
example:

Content-type: text/html

Content-type: image/webp

cgi: programming

The interaction between the client and Web server



cgi: variables

A CGI script has access to environment variables

associated to the request sent to the CGI program:

REQUEST_METHOD – HTTP method (GET, POST,...)

QUERY_STRING – data transmitted to the client

REMOTE_HOST, REMOTE_ADDR – client address

CONTENT_TYPE – content type as MIME (Media Type)

CONTENT_LENGTH – content length in bytes

cgi: variables

Additional variables

usually, generated by the Web server:

HTTP_ACCEPT – MIME types accepted by client (browser)

HTTP_COOKIE – data about cookies

HTTP_HOST – information regarding the host (client)

HTTP_USER_AGENT – information about the client

...and others

cgi: variables

Obtaining the values of the system's environment variables by invoking, through **GET**, the variable.cgi script stored on the Web server
(the program must have read and execution permissions)

```
#!/bin/bash
```

```
# Setting content type
```

```
echo "Content-type: text/plain";  
echo
```

```
# Executing 'set' command in Linux
```

```
# to display the environment variables
```

```
set
```

← → ↺ 🏠 https://profs.info.uaic.ro/~busaco/cgi/bash/variabile.cgi?date_de_intrare

```
GATEWAY_INTERFACE=CGI/1.1
GROUPS=()
HOSTNAME=profs.info.uaic.ro
HOSTTYPE=x86_64
HTTPS=on
HTTP_ACCEPT='text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8'
HTTP_ACCEPT_ENCODING='gzip, deflate, br'
HTTP_ACCEPT_LANGUAGE='en-GB,en;q=0.5'
HTTP_CONNECTION=keep-alive
HTTP_DNT=1
HTTP_HOST=profs.info.uaic.ro
HTTP_SEC_GPC=1
HTTP_UPGRADE_INSECURE_REQUESTS=1
HTTP_USER_AGENT='Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:85.0) Gecko/20100101 Firefox
IFS=$' \t\n'
MACHTYPE=x86_64-redhat-linux-gnu
OPTERR=1
OPTIND=1
OSTYPE=linux-gnu
PATH=/usr/local/bin:/usr/bin:/bin
PIPESTATUS=( [0]="0" )
PPID=25485
PS4='+ '
PWD=/home/thor/profs/busaco/html/cgi/bash
QUERY_STRING=date_de_intrare
REMOTE_ADDR=109.102.38.36
REMOTE_PORT=50606
REQUEST_METHOD=GET
REQUEST_SCHEME=https
REQUEST_URI='/~busaco/cgi/bash/variabile.cgi?date_de_intrare'
SCRIPT_FILENAME=/thor/profs/busaco/public_html/cgi/bash/variabile.cgi
SCRIPT_NAME=/~busaco/cgi/bash/variabile.cgi
SERVER_ADDR=85.122.23.20
SERVER_ADMIN=pimi@info.uaic.ro
SERVER_NAME=profs.info.uaic.ro
SERVER_PORT=443
SERVER_PROTOCOL=HTTP/1.1
SERVER_SIGNATURE=
SERVER_SOFTWARE='Apache/2.4.35 (IUS) OpenSSL/1.0.2k-fips mod_fcgid/2.3.9 PHP/7.2.34'
```

the result obtained
by the Web client

← → ↺ 🏠 https://profs.info.uaic.ro/~busaco/cgi/bash/variabile.cgi?date_de_intrare

```
GATEWAY_INTERFACE=CGI/1.1
GROUPS=()
HOSTNAME=profs.info.uaic.ro
HOSTTYPE=x86_64
HTTPS=on
HTTP_ACCEPT='text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8'
HTTP_ACCEPT_ENCODING='gzip, deflate, br'
HTTP_ACCEPT_LANGUAGE='en-GB,en;q=0.5'
HTTP_CONNECTION=keep-alive
HTTP_DNT=1
HTTP_HOST=profs.info.uaic.ro
HTTP_SEC_GPC=1
HTTP_UPGRADE_INSECURE_REQUESTS=1
HTTP_USER_AGENT='Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:85.0) Gecko/20100101 Firefox
IFS=$' \t\n'
MACHTYPE=x86_64-redhat-linux-gnu
OPTERR=1
OPTIND=1
OSTYPE=linux-gnu
PATH=/usr/local/bin:/usr/bin:/bin
PIPESTATUS=( [0]="0" )
PPID=25485
PS4='+ '
PWD=/home/thor/profs/busaco/html/cgi/bash
QUERY_STRING=date_de_intrare
REMOTE_ADDR=109.102.38.36
REMOTE_PORT=50606
REQUEST_METHOD=GET
REQUEST_SCHEME=https
REQUEST_URI='/~busaco/cgi/bash/variabile.cgi?date_de_intrare'
SCRIPT_FILENAME=/home/thor/profs/busaco/public_html/cgi/bash/variabile.cgi
SCRIPT_NAME=/~busaco/cgi/bash/variabile.cgi
SERVER_ADDR=85.122.23.20
SERVER_ADMIN=pimi@info.uaic.ro
SERVER_NAME=profs.info.uaic.ro
SERVER_PORT=443
SERVER_PROTOCOL=HTTP/1.1
SERVER_SIGNATURE=
SERVER_SOFTWARE='Apache/2.4.35 (IUS) OpenSSL/1.0.2k-fips mod_fcgid/2.3.9 PHP/7.2.34'
```

the result obtained
by the Web client

```
/* hello.c
   (compile with gcc hello.c -o hello.cgi) */
#include <stdio.h>

int main() {
    int msgs; /* number of messages */

    printf ("Content-type: text/html\n\n");

    for (msgs = 0; msgs < 10; msgs++) {
        printf ("<p>Hello, world!</p>");
    }

    return 0;
}
```

CGI programs written in C,
bash, Python generating
the same HTML content

```
#!/bin/bash
# hello.sh.cgi

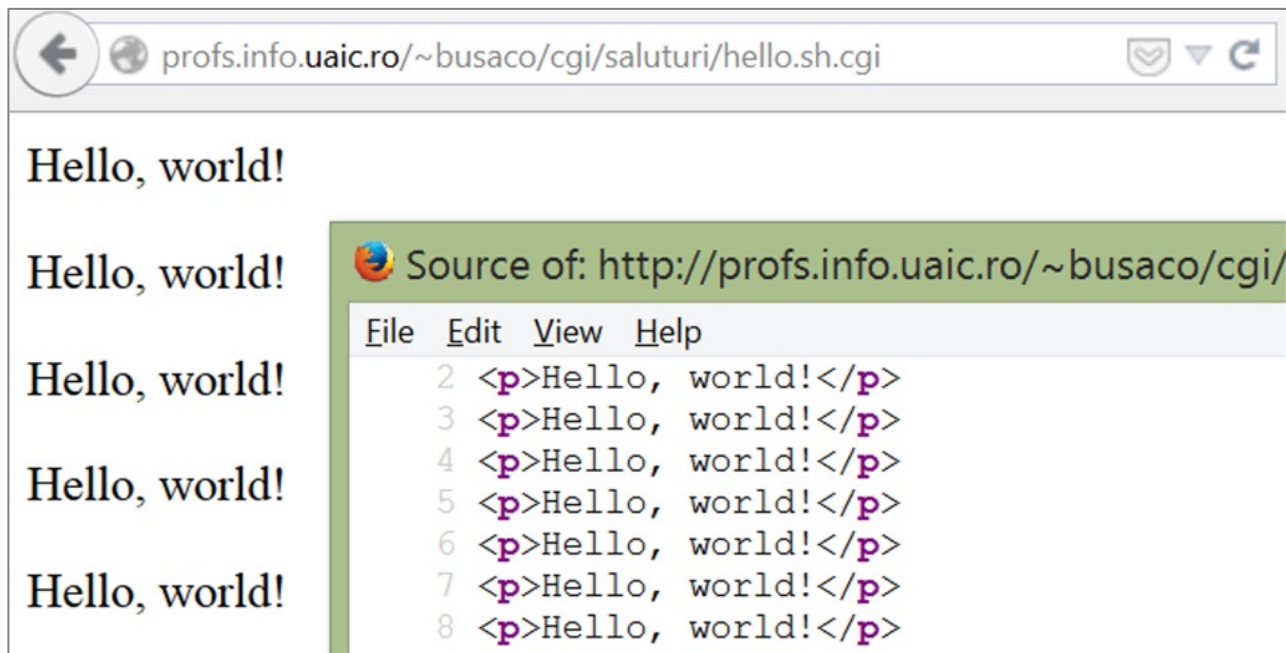
echo "Content-type: text/html"
echo

MESSAGES=0
while [ $MESSAGES -lt 10 ]
do
    echo "<p>Hello, world!</p>"
    let MESSAGES=MESSAGES+1
done
```

```
#!/usr/bin/python
# hello.py.cgi

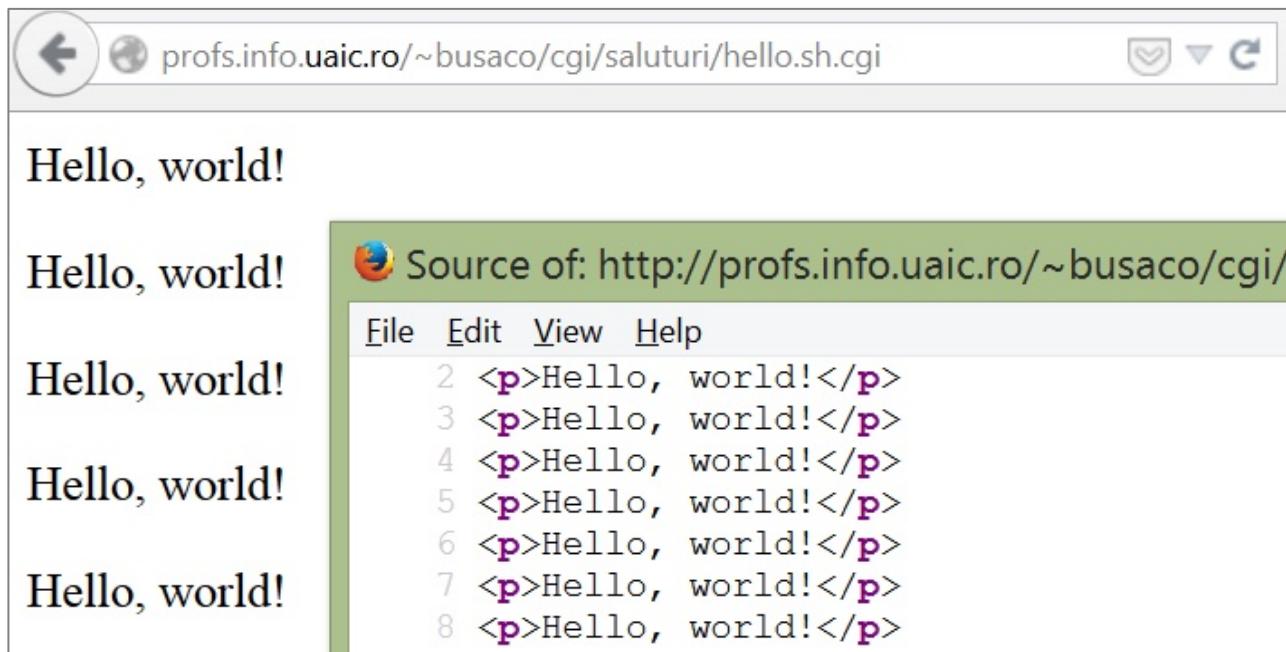
print "Content-type: text/html\n"

for messages in range (0, 10):
    print "<p>Hello, world!</p>"
```

the client – i.e. browser – receives the representation as
response – here, the HTML page –
generated by the CGI program invoked by the Web server

this representation is processed and, eventually, displayed
in a (zone of a) browser window



by experimenting other MIME types,
the browser displays the following:

```
<p>Hello, world!</p>
<p>Hello, world!</p>
<p>Hello, world!</p>
<p>Hello, world!</p>
<p>Hello, world!</p>
```

Content-type: **text/plain**

XML Parsing Error: junk after document element
Location: `http://profs.info.uaic.ro/~busaco/cgi/hello`
Line Number 2, Column 1:

```
<p>Hello, world!</p>
^
```

Content-type: **text/xml**

cgi: invocation

```
<form action="https://somewhere.info/cgi/get-max.cgi"
      method="GET">
  <p>Enter two numbers :
    <input type="text" name="no1" />
    <input type="text" name="no2" />
  </p>
  <input type="submit" value="Compute maximum" />
</form>
```

invocation from an interactive Web form
in this case, using the **GET** method

https://profs.info.uaic.ro/~busaco/cgi/max.html

Enter two numbers:

Compute maximum

special URL
in GET case

profs.info.uaic.ro/~busaco/cgi/get-max.cgi?no1=7&no2=4

Maximum of two numbers

$\max(7, 4) = 7$

cgi: invocation

For each form field, a **field_name=value** pair
– delimited by **&** – is generated and added to the URL
of the CGI script to be invoked on server

somewhere.info/cgi/get-max.cgi?no1=7&no2=4

cgi: invocation

Real-life examples:

alistapart.com/?s=web+design

www.youtube.com/watch?v=_0GWG0yCu-w#t=110

duckduckgo.com/?q=web%20development&ia=web

developer.mozilla.org/search?q=HTTP&topic=api

...  see first lecture

this URL is encoded – **URL encoding**

by convention, the space character included in the value of an input field in a Web form is replaced by “+”

cgi: invocation

The server will invoke the CGI script
passing the data received from the client
via **environment variables** – usually, in **GET** case
or
at **standard input (stdin)** – if **POST** is used

cgi: invocation

Data processing when **GET** method is used

data available in **QUERY_STRING** environment variable

cgi: invocation

Data processing when **POST** method is used

data read from stdin, the length in bytes
being specified by **CONTENT_LENGTH** variable

cgi: invocation

Data processing – **GET** and/or **POST**

in case of application servers or frameworks,
data is encapsulated into specific structures/types

ASP.NET (C# et. al) – **HttpRequest** class

Node.js (JavaScript) – **http.ClientRequest**

PHP – associative arrays: **\$_GET[]** **\$_POST[]** **\$_REQUEST[]**

Play (Java, Scala) – **play.api.mvc.Request**

Python – **cgi.FieldStorage**

GET vs. POST

GET method is used to get representations of the resources requested from the Web server

e.g., HTML documents, JPEG, PNG, or WebP images, SVG illustrations (XML), PDF resources, ZIP archives, etc.

 **the server state should not be modified**

GET vs. POST

GET method is used to get representations of the resources requested from the Web server

obtaining data with GET, the user can set a bookmark for further accesses to the Web resource (by using the URL of the generated representation)

e.g., scholar.google.com/scholar?as_ylo=2025&q=web+design&hl=en

GET vs. POST

POST method is used when the data transmitted to the server is large (e.g., upload of file content) or sensitive – typical example: passwords 😱

GET vs. POST

POST method is used when the data transmitted to the server is large (e.g., upload of file content) or sensitive – typical example: passwords 🤖

also, when the script invocation can produce a state change on the server:
adding a record, altering a file,...

cgi: support

The Web server should support
CGI script invocation

example:

Apache HTTP Server provides the **mod_cgi** module

cgi: ssi

CGI scripts could be directly invoked from an HTML document via SSI (Server Side Includes)

www.ssi-developer.net/ssi/

Apache: httpd.apache.org/docs/trunk/howto/ssi.html

NGINX: nginx.org/en/docs/http/nginx_http_ssi_module.html

cgi: fastcgi

FastCGI

an alternative to CGI focused on performance

implementations:

Apache HTTP Server – httpd.apache.org/mod_fcgid/

NGINX – nginx.org/en/docs/http/nginx_http_fastcgi_module.html

exercise

YouTube RO

Search

Q

Sign in

SPEAK TO ME

THE DARK SIDE OF THE MOON (50TH ANNIVERSARY)

PINK FLOYD

Wish You Were Here
Comfortably Numb
Hey You
Diamond
Time
Money
Learning To Fly
Proper Education
High Hopes

Marooned
The Great Gig In The Sky
Shine On You Crazy
Andy
What Do You Want From Me
Coming Back To Life
On The Turning Away
Shine on you crazy diamond
One Of These Days

2:10:28

Pink Floyd Collection
#Blues #Rock ...
Blues Relaxing Wave
3.1M views · 7 months ago

Pink Floyd | Timeless
Psychedelic Journey (Full ...
Savage Reel Lab

Inspector Console Debugger Performance Network Storage DOM Application

Filter URLs

All HTML CSS JS XHR Fonts Images Media WS Other

Disable Cache No Throttling

Status	Method	File	Proto...	Initiator	Type	Size	Duration
204	GET	watchtime?ns=yt&el=detail	HTTP/3	xhr	html	0 B	89 ms
200	POST	videoplayback?expire=1771	HTTP/3	fetch	vnd.yt-ump	562.08 kB	38 ms
200	POST	videoplayback?expire=1771	HTTP/3	fetch	vnd.yt-ump	129 B	8 ms
204	GET	watchtime?ns=yt&el=detail	HTTP/3	xhr	html	0 B	89 ms
204	POST	qoe?fmt=399&afmt=251&c		watch:1238 (fetch)			
204	GET	watchtime?ns=yt&el=detail	HTTP/3	xhr	html	0 B	89 ms
204	POST	log_event?alt=json		watch:1703 (xhr)			
204	HEAD	generate_204	HTTP/2	fetch	plain	0 B	63 ms
200	GET	data:text/plain;base64,Cg==		xhr	plain	4 B	0 ms

43 requests 2.70 MB / 2.54 MB transferred Finish: 4.94 min

Headers Cookies Request Response

Filter Headers

Block Resend

Response Headers (192 B)

Raw

HTTP/2 204
content-length: 0
cross-origin-resource-policy: cross-origin
date: Thu, 19 Feb 2026 07:06:26 GMT
alt-svc: h3=":443"; ma=2592000, h3-29=":443";
X-Firefox-Spdy: h2

Request Headers (1.231 kB)

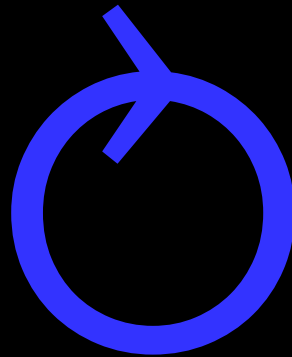
Raw

HEAD /generate_204 HTTP/2
Host: www.youtube.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac C
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br, zstd

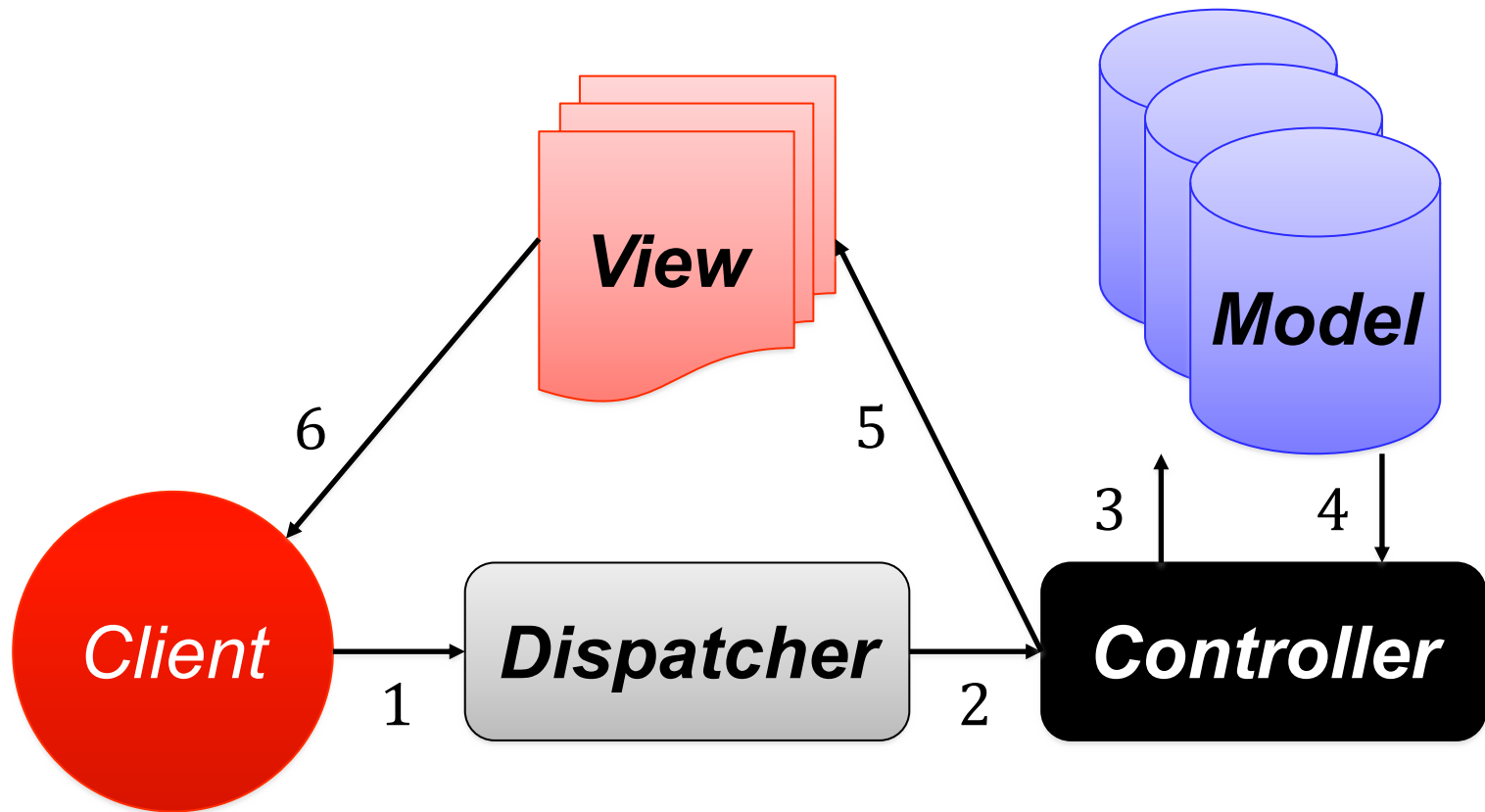


explain the HTTP request sequence

summary



HTTP (HyperText Transfer Protocol)
CGI (Common Gateway Interface) scripts



next episode: **Web programming**

cookies, Web sessions,

Web application servers, Web application architecture